

CSCI-SHU 360: Machine Learning

Final Competition Report - Fall 2025

Environmental Sound Classification with ResNet50

Hannah Huang
th2995@nyu.edu

Abstract

This project addresses environmental sound classification on short audio clips using an UrbanSound-style dataset with 10 classes. Raw waveforms are resampled, normalized to a fixed 4-second length, and converted into a 3-channel log-Mel representation (log-Mel + Δ + Δ²). The resulting time–frequency "image" is resized to 224×224 and fed into a ResNet50 backbone trained from scratch, with regularization and robustness improvements from mixup, SpecAugment, light spectrogram-domain perturbations, and 5-fold stratified cross-validation with fold-averaged test-time ensembling. Model selection is performed using a validation score combining accuracy and Macro-F1, and the final submission is produced by averaging softmax probabilities across folds.

1 Introduction

Environmental sound classification aims to recognize acoustic events (e.g., sirens, dog barks, engine noise) from short recordings. In this competition setup, each sample is a mono WAV clip labeled as one of 10 classes (classID 0–9). The task is a supervised multi-class classification problem: given an input waveform, the model predicts the corresponding classID. Submissions are evaluated on a held-out test set (Kaggle).

A common and effective approach is to represent audio as a time–frequency image (spectrogram) and apply strong image CNN architectures. Following this idea, I build a pipeline that (i) converts waveforms to log-Mel spectrogram features, (ii) uses a ResNet50-based classifier, and (iii) adopts a robust training strategy with augmentation, regularization, and stratified K-fold ensembling.

2 Method

2.1 Audio Preprocessing and Feature Extraction

2.1.1 Audio loading and length normalization

Each audio clip is loaded using `librosa.load` and resampled to a fixed sample rate of 22,050 Hz. To ensure a consistent input length across samples, each waveform is forced to a fixed duration of 4.0 s, i.e.

- target length = `sample_rate × duration = 22050 × 4 = 88200` samples.

If the clip is longer than 4 seconds, it is truncated to the first 88,200 samples. If shorter, it is zero-padded at the tail.

2.1.2 Log-Mel spectrogram computation

From the normalized waveform y, I compute a Mel spectrogram (power spectrogram mapped to Mel scale) using: `n_mels = 128, n_fft = 2048, hop_length = 512, fmin = 0 Hz, fmax = 8000 Hz, power = 2.0.`

Then I convert the Mel power spectrogram to log scale (dB):

$$logmel = power_to_db(melspec, ref = max)$$

2.1.3 3-channel feature construction (log-Mel + Δ + Δ²)

To enrich temporal dynamics and also match the standard 3-channel input convention for image CNNs, I build a 3-channel tensor:

- Channel 1: log-Mel spectrogram (dB) (logmel)
- Channel 2: first-order delta (Δ) computed by `librosa.feature.delta(logmel)`
- Channel 3: second-order delta (Δ²) computed by `librosa.feature.delta(logmel, order=2)`

These are stacked as: $x = stack([logmel, \Delta, \Delta^2]) \in \mathbb{R}^{3\times F\times T}$

2.1.4 Per-channel standardization

For numerical stability and to reduce distribution shift across clips, I apply per-channel z-score standardization:

$$x_c \leftarrow \frac{x_c - \mu_c}{\sigma_c + 10^{-6}}, c \in \{1, 2, 3\}$$

where μ_c and σ_c are computed over all (F, T) locations within channel c.

2.1.5 Resize to 224×224 (image-like input)

The spectrogram tensor has shape (3, F, T) with F = 128 and T determined by FFT/hop and duration. To use ResNet50 directly, I resize each sample to (3, 224, 224) via bilinear interpolation (`torch.nn.functional.interpolate`). This allows the model to treat each example as a standard image input while retaining global time–frequency structure.

2.2 Model Architecture: ResNet50 Classifier

2.2.1 Backbone

I use ResNet50 (`torchvision.models.resnet50`) with `weights=None`, meaning the entire network is trained from scratch on this dataset. The input is a resized spectrogram "image" (3, 224, 224), and the output is a 10-dimensional logit vector.

2.2.2 Classification head

The original fully-connected layer is replaced by:

- `Dropout(p=0.2)`
- `Linear(in_features, 10)`

The linear layer is initialized using Xavier uniform initialization, and biases are zero-initialized. Dropout adds regularization, which is especially important because training is from scratch and the dataset is not extremely large.

2.3 Training Strategy: Augmentation and Regularization

2.3.1 Mixup (batch-level)

I apply mixup to each mini-batch during training. Given two samples (x_i, y_i) and (x_j, y_j), I generate: $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$, where $\lambda \sim Beta(\alpha, \alpha)$ with $\alpha = 0.3$.

The loss is a convex combination of two cross-entropies:

$$\mathcal{L} = \lambda CE(f(\tilde{x}), y_i) + (1 - \lambda)CE(f(\tilde{x}), y_j).$$

Mixup smooths decision boundaries and reduces overfitting, which is beneficial for spectrogram classification where classes can be acoustically similar.

2.3.2 SpecAugment (spectrogram masking)

I apply SpecAugment during training by masking random rectangles. SpecAugment uses 2 masks with max widths: freq ≤ 24 bins and time ≤ 48 frames.

For each mask, a random band in frequency and time is set to zero.

2.3.3 Light spectrogram perturbations

In addition to SpecAugment, I use lightweight perturbations (training only):

- **Random gain**: multiply the spectrogram by a factor corresponding to [-6, +6] dB
- **Random Gaussian noise**: add noise with scale in [0.01, 0.05]
- **Random time shift**: roll along the time axis by a small shift in [-20, 20] frames

2.3.4 Loss function: class weighting + label smoothing

I use CrossEntropyLoss with:

- label smoothing = 0.05
- class weights to emphasize minority classes

Weights are computed from per-fold class counts n_k as: $w_k = \frac{1}{\sqrt{n_k}}$

2.4 Optimization, Data Loading, Cross-Validation, and Ensembling

2.4.1 Optimizer and scheduler

- Optimizer: AdamW
  - learning rate = 7e-4
  - weight decay = 1e-4
- Scheduler: OneCycleLR
  - `pct_start = 0.15`
  - `div_factor = 25`
  - `final_div_factor = 10^4`
  - stepped every iteration (per batch)

OneCycleLR typically provides fast convergence and better generalization by increasing LR early and annealing later.

2.4.2 Gradient clipping and mixed precision

I apply gradient clipping (max norm = 1.0) for stability, and enable AMP (autocast + GradScaler) on CUDA to improve speed and memory efficiency.

2.4.3 DataLoader settings

I use batch size 48 and multi-worker DataLoader with GPU pinning when available to improve throughput.

2.4.4 5-fold stratified cross-validation

I use StratifiedKFold (n=5) on `classID` to maintain similar class distributions in each fold.

For each fold:

1. Train a ResNet50 model on 4 folds
2. Validate on 1 fold
3. Save the checkpoint with the best validation metric

2.4.5 Validation metric for checkpointing

During training I select checkpoints using a combined score: Score = 0.8 · Accuracy + 0.2 · Macro-F1.

2.4.6 Test-time ensembling (fold averaging)

For inference on the test set, I compute softmax probabilities from each fold model and average them:

$$p_{final} = \frac{1}{K}\sum_{k=1}^{K} p^{(k)}$$

3 Experiments

3.1 Stronger CNN Baseline

I first built a stronger CNN baseline on log-Mel spectrograms using several Conv–BN–ReLU–MaxPool blocks and a small FC head, with z-score standardized inputs. I trained on folds 1–7 and validated on fold 8, mainly tuning Adam (lr ≈ 1e-3), ~8 epochs (longer runs overfit), and moderate SpecAugment + label smoothing (0.1) with ReduceLROnPlateau. This baseline achieved mid-70% validation accuracy and a Kaggle score of ~0.77, suggesting the preprocessing worked but a stronger backbone and more robust validation were needed.

3.2 Transition to ResNet34

To improve generalization, I switched to a ResNet34 backbone and adopted 5-fold stratified cross-validation with fold-averaged ensembling on the test set. I compared learning-rate strategies (fixed LR, ReduceLROnPlateau, OneCycleLR) and found OneCycleLR produced smoother training and better mean validation accuracy. I also tuned training length (short runs underfit some folds; very long runs overfit) and used moderate SpecAugment plus mixup (Beta with $\alpha \approx 0.4$), since overly aggressive masking/mixup could hurt performance. This transition improved performance to around 0.80 and reduced sensitivity to a single split.

3.3 Final ResNet50 Pipeline

The final model is a ResNet50 with 5-fold stratified CV and probability averaging across folds for inference. Compared to earlier versions, the main refinements were: (i) a 3-channel spectrogram input (log-Mel, Δ, Δ^2) with per-channel standardization and resizing to (3,224,224) to match ResNet's native interface; (ii) stronger but controlled regularization—Dropout(0.2), label smoothing (0.05), mixup (α=0.3), SpecAugment (2 masks; freq≤24; time≤48), plus light spectrogram perturbations (gain/noise/time shift); (iii) more stable optimization using AdamW (lr=7e-4, wd=1e-4), OneCycleLR, gradient clipping (1.0), and AMP on CUDA; and (iv) class-weighted cross-entropy (weights $\propto 1/\sqrt{\text{count}}$) with checkpoint selection based on a combined metric. These changes were aimed at stabilizing deeper-from-scratch training and improving minority-class performance while maintaining strong overall accuracy.

4 Conclusion

This report presents an end-to-end pipeline for environmental sound classification that transforms fixed-length audio clips into a 3-channel log-Mel representation (log-Mel, Δ, Δ^2) and trains a ResNet50 model from scratch. Generalization is improved through mixup (α=0.3), SpecAugment, lightweight spectrogram perturbations (gain/noise/time shift), label smoothing (0.05), Dropout (0.2), and class-weighted cross-entropy. Training is stabilized with AdamW + OneCycleLR, gradient clipping, and AMP on CUDA, and final predictions are obtained via 5-fold stratified cross-validation with fold-averaged probability ensembling at test time.

On the Kaggle test set, the final submission achieves a public leaderboard score of 0.87.

5 Kaggle Score Screenshot

